# CS-4110 HPC with GPUs

Dr Imran Ashraf
Associate Professor
CS, NUCES-FAST, Islamabad
C-205-E

# This week

- Communication optimization
- Memory Assignment
  - Constant Memory
  - Shared memory

# CPU-GPU communication optimizations

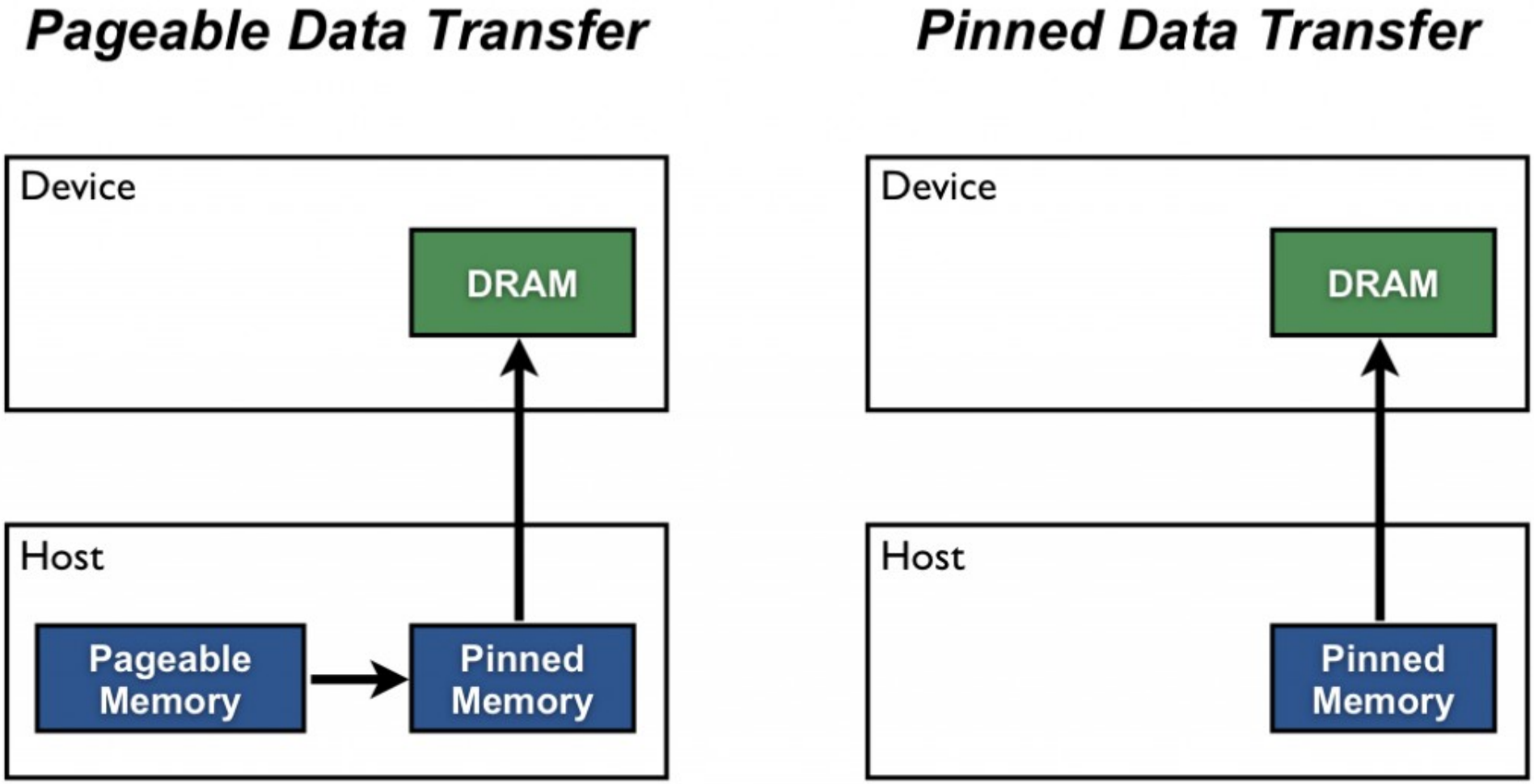
- CPU-GPU data transfer is expensive

- Avoid un-necessary moves
- Minimize necessary moves
- Batch small transfers to one large transfer
- Use pinned memory
- Overlap data transfer with kernel execution
  - Streams

# CPU-GPU communication optimizations

- Any examples in vecOp code?
  - Input array
  - Initialization

# Peggable vs Pinned Memory



# Pageable Memory

- Pageable memory is the default memory allocation type in CUDA.
- It is a virtual memory space that is mapped to physical memory pages.
- The CUDA runtime system handles the mapping between virtual and physical memory addresses.

# Pinned Memory

- Pinned memory is explicitly pinned to a specific physical memory location.
- It is allocated from the host memory space.
- Pinned memory is not subject to page faults.
- Pinned memory is not swapped out to disk.

- Pinned memory can provide better memory access performance due to its explicit allocation and lack of page faults.

# Peggable vs Pinned Memory

```
h_aPageable = (float*)malloc(bytes); // host pageable
```

vs

```
cudaMallocHost((void**)&h_aPinned, bytes);  // host pinned
```

# Peggable vs Pinned Memory

```
Device: NVS 4200M
Transfer size (MB): 16

Pageable transfers
  Host to Device bandwidth (GB/s): 2.308439
  Device to Host bandwidth (GB/s): 2.316220

Pinned transfers
  Host to Device bandwidth (GB/s): 5.774224
  Device to Host bandwidth (GB/s): 5.958834
```

Reference: https://devblogs.nvidia.com/how-optimize-data-transfers-cuda-cc/

# Bandwidth Comparison

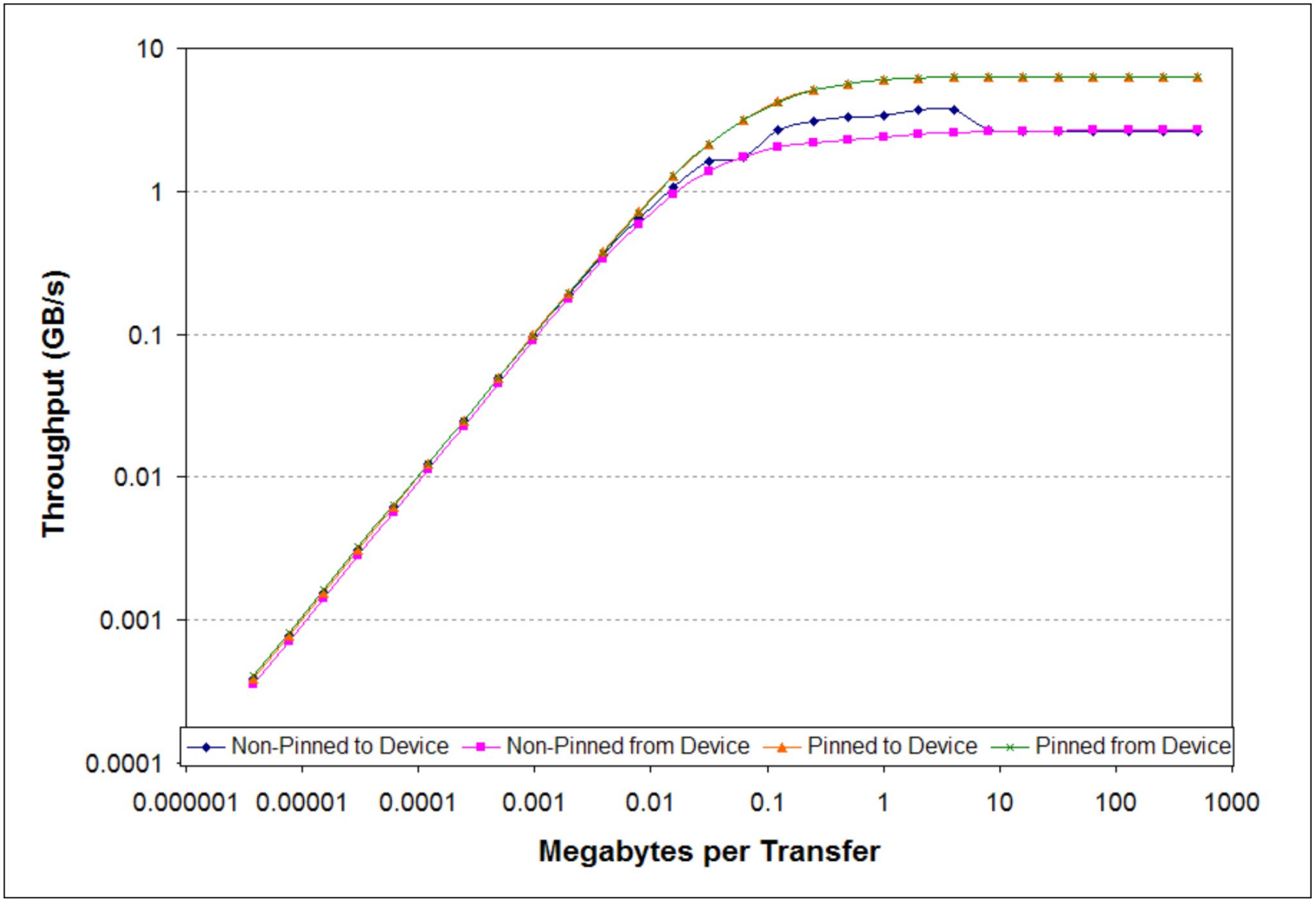
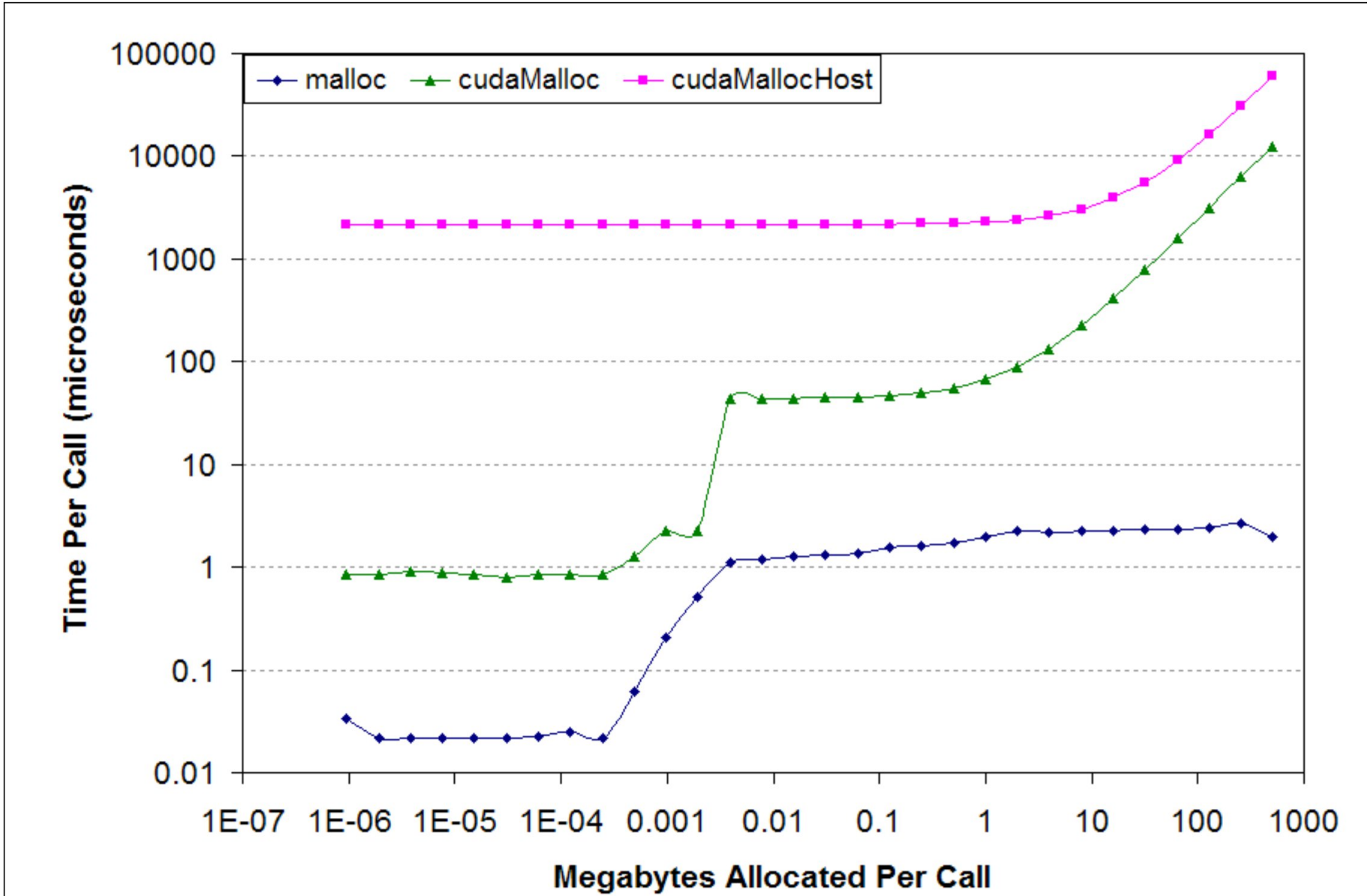
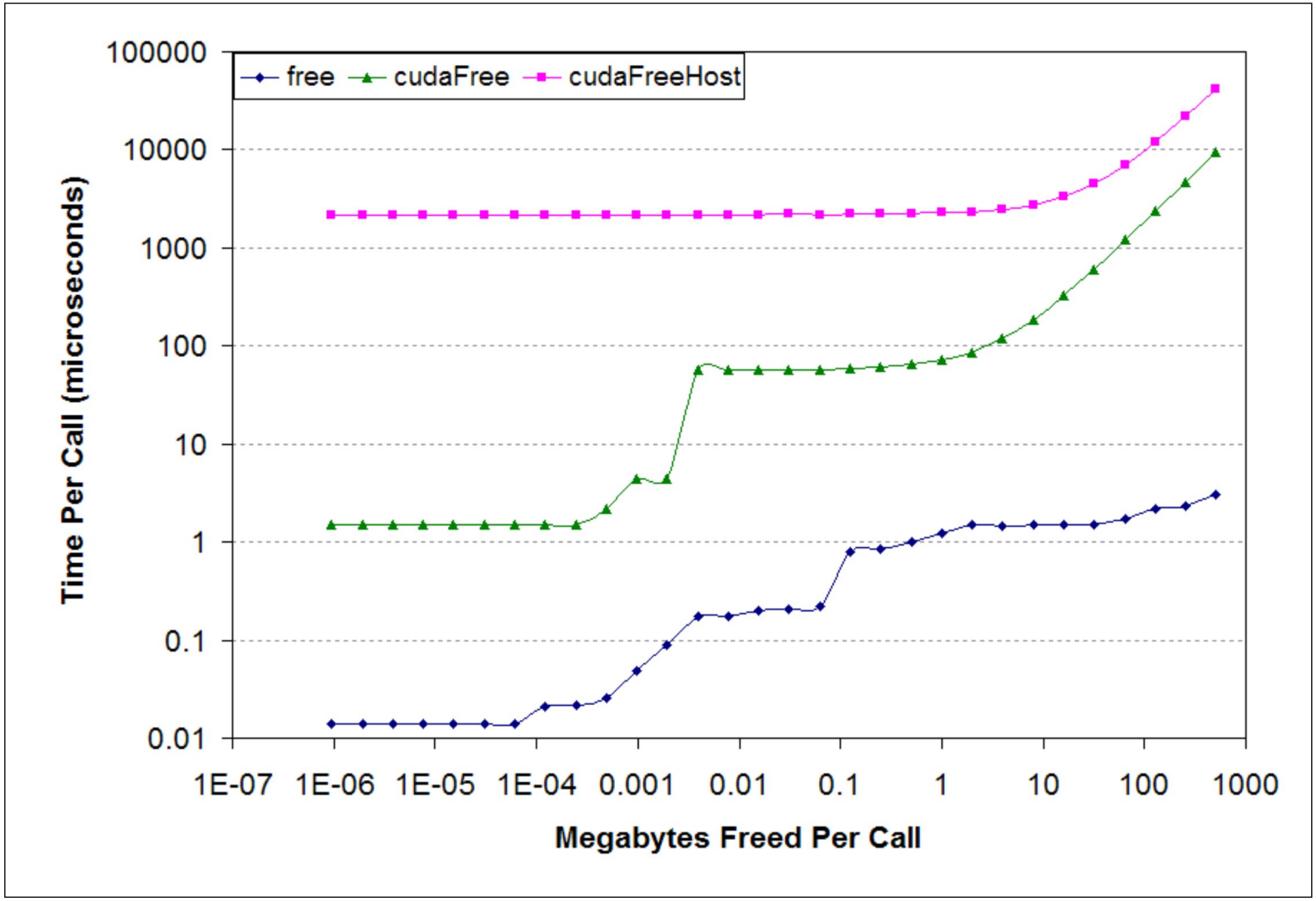



**Figure 2: Transfer throughput as a function of transfer size. Note the logarithmic scales.**

# Allocation Time Comparison

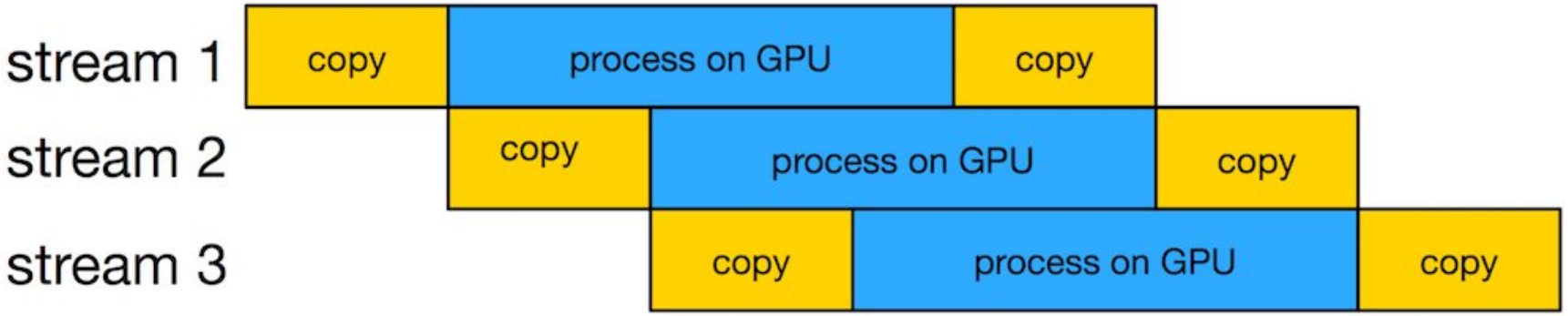


# Free Time Comparison

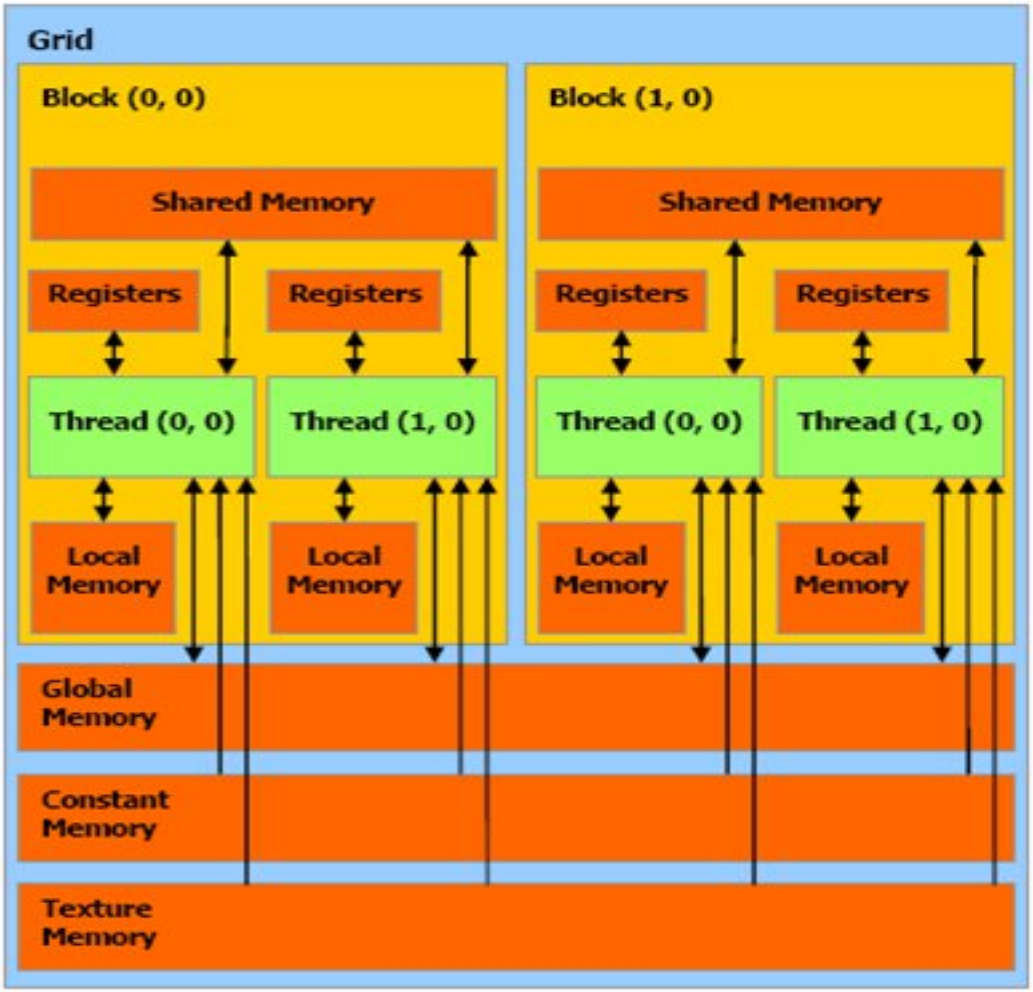


# Streams

- A sequence of operations that execute in issue-order on the GPU
  - CUDA operations in different streams may run concurrently
  - CUDA operations from different streams may be interleaved



# GPU Memory Hierarchy




Speed (Fast to slow):

1. Register file
2. Shared Memory
3. Constant Memory
4. Texture Memory
5. Global Memory

# CUDA: Declaring memory

| Declaration | Memory | Scope | Lifetime |
|---|---|---|---|
| int v | register | thread | thread |
| int vArray[10] | local | thread | thread |
| __shared__ int sharedV | shared | block | block |
| __device__ int globalV | global | grid | application |
| __constant__ int constantV | constant | grid | application |

# Constant Memory

- Global Scope → Declared outside kernel with __constant__.
- Read-Only in Kernel → Can be read by threads but cannot be modified inside the kernel.
- Efficient for Broadcasts → Best used when many threads read the same data.
- Limited to 64KB → Use it for small, frequently used data.

# Declaring Constant Memory

- __constant__ int constantData[256];

- int h_data[N] = {1, 2, 3, 4};
- cudaMemcpyToSymbol(constantData, h_data, N * sizeof(int));

# Shared Memory

- High-speed memory located inside each SM (Streaming Multiprocessor).
- Shared among all threads in a block but not across blocks.
- Much faster than global memory but limited in size (typically 48KB per SM).
- Used for data reuse, reducing global memory accesses and improving performance.

# Shared Memory Usage

- Frequent Data Reuse:
  - When multiple threads in a block need to access the same data multiple times.
  - Reduces redundant global memory accesses, improving performance.
- Parallel Reduction & Prefix Sum
- Stencil Computations (e.g., Convolution, Finite Difference Methods)
- Sorting Algorithms (e.g., Bitonic Sort, Radix Sort)
- Matrix Multiplication (Tiling Optimization)
- Coalescing Global Memory Accesses
- Atomic Operations on Shared Data

# Allocate shared memory

- Static

  __shared__ int sharedArray[256];

- Dynamic
  - extern __shared__ int sharedArray[];
  - kernelFunction<<<1, size, size * sizeof(int)>>>(d_in, d_out, size);

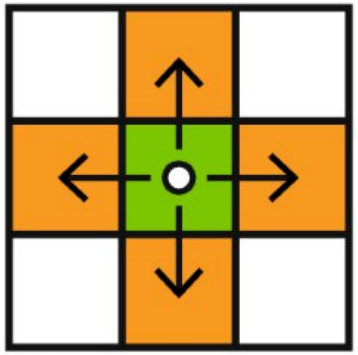
# Example 00: Shared Memory

```
__global__ void simpleSharedMemory(int *d_in, int *d_out) {
    __shared__ int sharedData[N];  // Declare shared memory

    int tid = threadIdx.x;

    // Copy data from global memory to shared memory
    sharedData[tid] = d_in[tid];

    __syncthreads();  // Ensure all threads have copied data

    // Modify shared memory (increment)
    sharedData[tid] += 1;

    __syncthreads();  // Ensure all modifications are complete

    // Copy data back to global memory
    d_out[tid] = sharedData[tid];
}
```

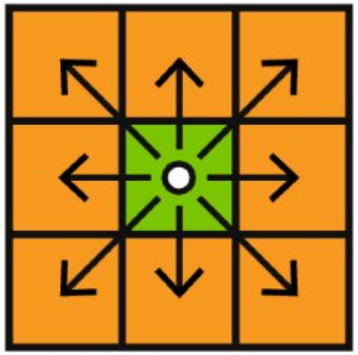
# Example 01: Shared Memory

```
__global__ void staticReverse(int *d, int n)
{
  __shared__ int s[64];
  int t = threadIdx.x;
  int tr = n-t-1;
  s[t] = d[t];
  // Will not conttinue until all threads completed.
  __syncthreads();
  d[t] = s[tr];
}
```

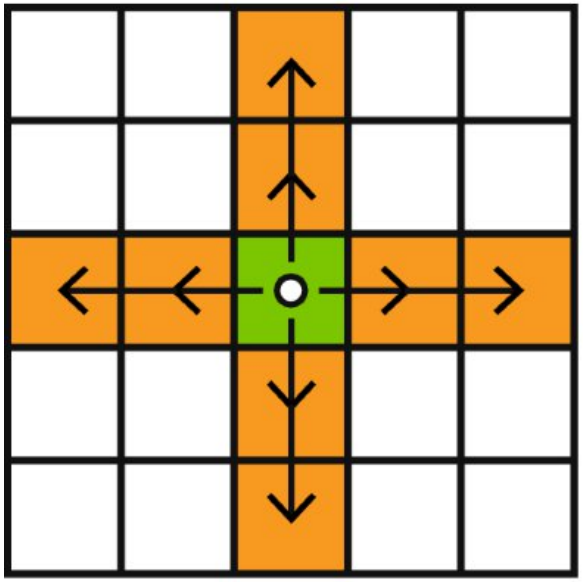
# Stencil Computation



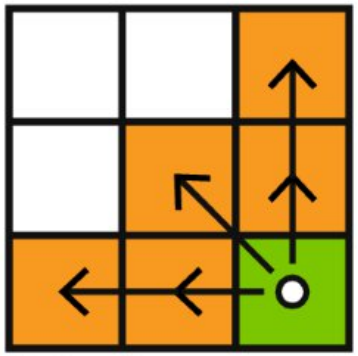
2D, 4-point compact
(Jacobi)



2D, 9-point compact



2D, 9-point non-compact



2D, 5-point asymmetric
non-compact

# Stencil Computation

- A technique where each element is updated using neighboring elements.
- Common in image processing, physics simulations, and PDE solvers.
- Example Formula (3-point stencil):
  $$d\_out[i] = 0.25 \times d\_in[i-1] + 0.50 \times d\_in[i] + 0.25 \times d\_in[i+1]$$

# Stencil Computation: Naïve Approach

- Each thread reads required values from global memory directly.
- No data reuse, leading to high global memory latency.
- Redundant memory accesses occur when neighboring threads read the same values.

# Stencil Computation: Naïve Approach

```
__global__ void naiveStencil(float *d_out, float *d_in, int
n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx > 0 && idx < n - 1) { // Avoid out-of-bounds
            d_out[idx] = 0.25f * d_in[idx - 1] + 0.5f * d_in[idx]
+  0.25f * d_in[idx + 1];
    }
}
```

# Stencil Computation: Naïve Approach

- Each thread accesses 3 global memory locations, causing redundant reads.
- Global memory is slow (~400-600 cycles per access).
- No caching mechanism, increasing memory bandwidth usage.

# Stencil Computation: Halo Cells

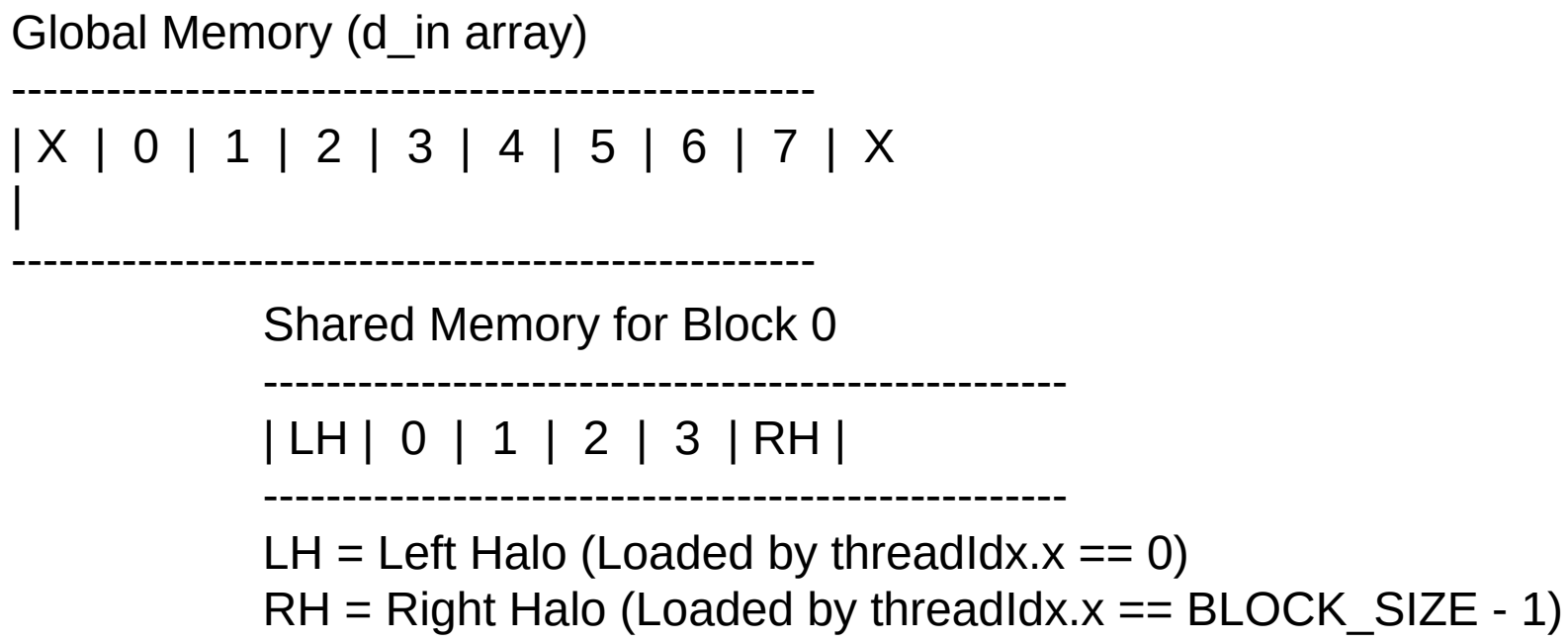
- Handling Boundary Conditions
  - Threads at block edges depend on neighboring blocks' data, which may be unavailable in global memory at the moment.
  - Halo cells store extra data from neighboring blocks.

# Stencil Computation: Halo Cells

- Reducing Memory Access Overhead
  - Instead of each thread fetching redundant values, shared memory allows data reuse.

# Stencil Computation: Halo Cells

- Performance Benefits
  - Threads can access local memory, reducing memory contention.



# Stencil Computation: v1

```
__global__ void stencilKernel(float *d_out, float *d_in, int n) {
    __shared__ float s_data[BLOCK_SIZE + 2];  // Shared memory with halo

    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int tid = threadIdx.x;

    // Load data into shared memory (including halos)
    if (idx < n) {
        s_data[tid + 1] = d_in[idx];  // Load main data

        if (tid == 0 && idx > 0)  // First thread loads left halo
            s_data[0] = d_in[idx - 1];

        if (tid == blockDim.x - 1 && idx < n - 1)  // Last thread loads right halo
            s_data[tid + 2] = d_in[idx + 1];

        __syncthreads();  // Ensure all data is loaded before computation
```

# Stencil Computation: v1

```
__global__ void stencilKernel(float *d_out, float *d_in, int n) {

        ....

        __syncthreads();  // Ensure all data is loaded before computation

        // Perform stencil computation (skip boundary threads)
        if (idx > 0 && idx < n - 1) {
            d_out[idx] = 0.25f * s_data[tid] +
                         0.5f * s_data[tid + 1] +
                         0.25f * s_data[tid + 2];
        }
    }
}
```

# Stencil Kernel: v2

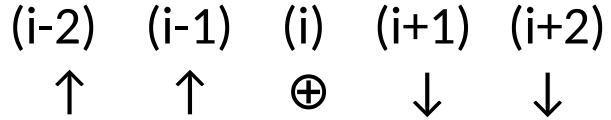
```
__global__ void stencil_1d(int *in, int *out) {
  __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
  int gindex = threadIdx.x + blockIdx.x * blockDim.x;
  int lindex = threadIdx.x + RADIUS;

  // Read input elements into shared memory
  temp[lindex] = in[gindex];
  if (threadIdx.x < RADIUS) {
    temp[lindex - RADIUS] = in[gindex - RADIUS];
    temp[lindex + BLOCK_SIZE] =
      in[gindex + BLOCK_SIZE];
  }

 // Synchronize (ensure all the data is available)
    __syncthreads();
```

# Stencil Kernel: v2

```
  // Apply the stencil
  int result = 0;
  for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
    result += temp[lindex + offset];

  // Store the result
  out[gindex] = result;
}
```

# 1D 5-Point Stencil Computation

Each element (i) is updated using its two left and two right neighbors:



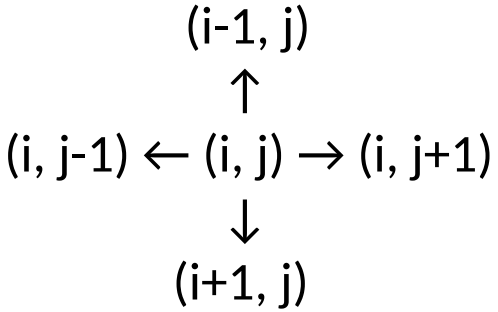

d_out[i] = 0.125×d_in[i−2]+
0.250×d_in[i−1]+
0.250×d_in[i+1]+
0.125×d_in[i+2]+
0.250×d_in[i]

# 2D Stencil Computation

- The same concept applies to 2D grids.
- Threads load halo rows and columns in shared memory.
- Used in heat diffusion, convolution filters, and PDE solvers.

# 2D 5-Point Stencil Computation

Each cell depends on its left, right, top, and bottom neighbors while updating the value of (i, j).




$$d\_out[i][j] = 0.125 \times d\_in[i-1][j] + 0.125 \times d\_in[i+1][j] + 0.125 \times d\_in[i][j-1] + 0.125 \times d\_in[i][j+1] + 0.500 \times d\_in[i][j]$$

# 2D 5-Point Stencil Computation

```
__global__ void stencil2D(float *d_out, float *d_in, int width, int height) {
    __shared__ float tile[BLOCK_SIZE + 2][BLOCK_SIZE + 2];

    int x = threadIdx.x + blockIdx.x * blockDim.x;
    int y = threadIdx.y + blockIdx.y * blockDim.y;
    int tx = threadIdx.x + 1;
    int ty = threadIdx.y + 1;
```

# 2D 5-Point Stencil Computation

```
// Load the main cell
if (x < width && y < height) {
    tile[tx][ty] = d_in[y * width + x];

    // Load halos
    if (threadIdx.x == 0 && x > 0)
        tile[0][ty] = d_in[y * width + (x - 1)];
    if (threadIdx.x == blockDim.x - 1 && x < width - 1)
        tile[BLOCK_SIZE + 1][ty] = d_in[y * width + (x + 1)];
    if (threadIdx.y == 0 && y > 0)
        tile[tx][0] = d_in[(y - 1) * width + x];
    if (threadIdx.y == blockDim.y - 1 && y < height - 1)
        tile[tx][BLOCK_SIZE + 1] = d_in[(y + 1) * width + x];

    __syncthreads();
```

# 2D 5-Point Stencil Computation

```
    // Apply the 5-point stencil
    if (x > 0 && x < width - 1 && y > 0 && y < height - 1) {
      d_out[y * width + x] = 0.125f * (tile[tx - 1][ty] + tile[tx + 1][ty] +
                                       tile[tx][ty - 1] + tile[tx][ty + 1]) +
                             0.5f * tile[tx][ty];
    }
  }
}
```

# Useful Reading

- https://lwn.net/Articles/250967/
- https://people.cs.clemson.edu/~dhouse/courses/405/papers/optimize.pdf